Universidad Nacional Sede Regional Brunca

Campus Pérez Zeledón / Campus Coto
EIF205 - Arquitectura de Computadores
II Proyecto (Ensamblador) - II Ciclo 2026
Prof: Daniel Granados M., Gabriel Núñez M.
Valor Total: 12 %

Presentación del proyecto.

Mastermind (conocido también como «Toros y Vacas», «Bulls and Cows» o «Picas y Fijas») es un juego de deducción lógica en el que un jugador debe descubrir un código secreto de colores mediante intentos sucesivos, guiándose únicamente por la retroalimentación numérica que el programa devuelve después de cada intento.
En este proyecto usted y su pareja implementarán una versión completa del juego en lenguaje ensamblador x86 de 16 bits, ejecutándose en modo gráfico y con una identidad visual propia construida a partir de sprites animados diseñados por ustedes.
El proyecto persigue tres cosas a la vez: (a) que resuelvan un problema algorítmico no trivial con las herramientas limitadas del ensamblador; (b) que manipulen directamente el hardware de video y el teclado a través del BIOS y de la memoria mapeada, entendiendo lo que ocurre por debajo de cualquier biblioteca gráfica; y (c) que puedan explicar y defender por escrito lo que programaron.

> La lógica del juego es idéntica para todas las parejas. Lo que debe ser distinto en cada entrega es la personalización: el tema visual, los sprites, las animaciones y la presentación. Dos proyectos con la misma temática y los mismos sprites serán tratados como copia.



Figura 1: Presentación

Objetivo general

Diseñar e implementar, en lenguaje ensamblador x86 de 16 bits, un juego de Mastermind con interfaz gráfica basada en sprites animados, aplicando el manejo directo de la memoria de video, las interrupciones del BIOS y las estructuras de control y datos propias del ensamblador.

Objetivos específicos

- Traducir un algoritmo de comparación con conteo de repeticiones (toros y vacas) a instrucciones de máquina, usando registros, banderas y saltos condicionales.
- Programar el adaptador de video en modo 13h escribiendo directamente en el segmento A000h, y calcular desplazamientos de píxel a partir de coordenadas (x, y).
- Manipular la paleta de 256 colores del VGA mediante los puertos 3C8h y 3C9h con la instrucción OUT.
- Implementar rutinas reutilizables de dibujo de sprites con color transparente, y animarlas mediante control de tiempo.
- Capturar entrada de teclado con INT 16h, distinguiendo códigos ASCII de códigos de exploración para las teclas extendidas.
- Organizar el programa en procedimientos con paso de parámetros, manejo correcto de la pila y preservación de registros.
- Documentar y defender técnicamente las decisiones de implementación.

Reglas del juego

El programa debe generar un código secreto de N posiciones de forma aleatoria, tomando colores de una paleta de C colores disponibles. El jugador dispone de un número limitado de intentos para descubrirlo.

Niveles de dificultad

El juego debe ofrecer los tres niveles siguientes, seleccionables desde el menú principal:

Nivel	Posiciones (N)	Colores (C)	Repetición de colores	Intentos
Fácil	4	6	No permitida	12
Normal	4	6	Permitida	10
Difícil	5	8	Permitida	10

Mecánica de una jugada

1. El jugador arma un intento seleccionando un color para cada una de las N posiciones.
2. El intento solo se evalúa cuando todas las posiciones están llenas y el jugador confirma con ENTER. Un intento incompleto no consume turno.
3. El programa evalúa el intento y devuelve dos números: la cantidad de toros y la cantidad de vacas.
4. El intento evaluado se conserva visible en el tablero junto con su retroalimentación, de modo que el jugador pueda razonar sobre el historial completo.

5. El turno avanza. Si el jugador acierta el código completo, gana. Si agota los intentos sin acertar, pierde y el programa revela el código secreto.

Toros y vacas

Toro (bull): color correcto en la posición correcta.
Vaca (cow): color correcto, pero en una posición equivocada.
La regla más importante es que cada ficha del código secreto puede contar una sola vez. Un color que ya fue contado como toro no puede volver a contarse como vaca, y si el secreto tiene dos fichas rojas pero el intento tiene tres, solo dos de ellas pueden generar retroalimentación.

Secreto	Intento	Toros	Vacas	Por qué
R V A M	R A V M	2	2	R y M están en su lugar; V y A están pero cruzados.
R R V A	R V R R	1	2	Solo la primera R es toro. Del intento, la V y una R más coinciden; la tercera R del intento ya no tiene con qué emparejarse.
R V A M	N B N B	0	0	Ningún color del intento aparece en el secreto.
A A A A	A A R R	2	0	Dos toros; las R sobrantes no coinciden con nada.

Los cuatro casos de la Tabla 2 serán ejecutados durante la revisión. Un programa que falle en el segundo o el cuarto caso pierde la totalidad del criterio «corrección del algoritmo». Deben poder ingresar el código secreto de manera manual para hacer dichas pruebas.



Figura 2: Menú

Requisitos funcionales obligatorios

Todos los requisitos de esta sección son de cumplimiento obligatorio. Su ausencia se descuenta directamente del rubro «Lógica y funcionamiento».

ID	Requisito	Criterio de aceptación
RF-01	Pantalla de presentación con el nombre del juego, la temática elegida y los integrantes.	Contiene al menos un sprite animado y espera una tecla para continuar.
RF-02	Menú principal navegable por teclado.	Opciones: Jugar, Nivel de dificultad, Instrucciones, Créditos, Salir. El elemento seleccionado se resalta visualmente.
RF-03	Pantalla de instrucciones dentro del juego.	Explica las reglas y el significado de toros y vacas con apoyo gráfico, no solo texto.
RF-04	Generación pseudoaleatoria del código secreto.	La semilla proviene del contador de tics del sistema (INT 1Ah, función 00h). Dos partidas consecutivas no producen el mismo código.
RF-05	Respeto de la restricción de repetición según el nivel.	En nivel Fácil el código secreto no contiene colores repetidos.
RF-06	Selección de color por posición mediante teclado.	Flechas ← → mueven la posición activa; flechas ↑ ↓ o teclas 1..C cambian el color; ENTER confirma; BACKSPACE borra la posición actual.
RF-07	Validación del intento antes de evaluarlo.	Un intento incompleto no consume turno y muestra un aviso en pantalla.
RF-08	Cálculo correcto de toros y vacas.	Supera los cuatro casos de la Tabla 2.
RF-09	Tablero con el historial completo de intentos.	Todos los intentos previos con su retroalimentación permanecen visibles simultáneamente.
RF-10	Indicadores de estado.	Intento actual, intentos restantes y nivel de dificultad visibles durante toda la partida.
RF-11	Detección de victoria y de derrota.	Victoria al obtener N toros; derrota al agotar los intentos. En la derrota se revela el código secreto.
RF-12	Pantalla de fin de partida con animación diferenciada.	La animación de victoria y la de derrota son visiblemente distintas.
RF-13	Reinicio sin salir del programa.	Desde el fin de partida se puede iniciar otra partida o volver al menú.
RF-14	Salida ordenada.	Restaura el modo de video original (guardado con INT 10h función 0Fh antes de cambiarlo) y devuelve el control a DOS con INT 21h función 4Ch.

Funcionalidad opcional (+5 % según complete todas las tareas)

- Contador de tiempo de partida usando el contador de tics del BIOS.
- Tabla de mejores puntajes persistida en un archivo mediante INT 21h (funciones 3Ch, 3Dh, 3Fh, 40h, 3Eh).
- Efectos de sonido con el altavoz interno (puerto 61h y temporizador 8253/8254 en el puerto 43h).
- Modo inverso: el jugador propone el código y el programa lo deduce aplicando una estrategia de descarte.
- Modo dos jugadores por turnos en la misma máquina.
- Transiciones de pantalla mediante manipulación progresiva de la paleta (fade in / fade out).

Requisitos gráficos y de personalización

Este es el bloque que diferencia una entrega de otra y el que concentra el 20 % de la nota. No se aceptan fichas dibujadas como cuadrados o círculos de color plano: cada «color» del juego debe estar representado por un sprite propio, diseñado por ustedes.

ID	Requisito	Criterio de aceptación
RG-01	Modo gráfico 13h.	El programa entra al modo 13h con INT 10h función 00h y escribe los píxeles directamente en el segmento A000h. Se prohíbe usar INT 10h función 0Ch para el dibujo masivo.
RG-02	Rutina genérica de dibujo de sprites.	Un único procedimiento DibujarSprite recibe puntero a datos, ancho, alto y coordenadas. No se acepta una rutina distinta y repetida para cada figura.
RG-03	Color transparente.	El índice de color 0 del sprite no se escribe en pantalla, de modo que los sprites no arrastran un rectángulo de fondo.
RG-04	Sprites propios para cada color del juego.	Mínimo 6 sprites distintos (8 en dificultad difícil) de al menos 16 × 16 píxeles, distinguibles entre sí no solo por el tono sino por la forma.
RG-05	Animación por cuadros.	Al menos tres elementos animados con un mínimo de 2 cuadros cada uno: la mascota de la pantalla de presentación, la ficha seleccionada en el tablero y la animación de fin de partida.
RG-06	Control de tiempo de la animación.	La velocidad de la animación no depende de la velocidad de la CPU: se controla con el contador de tics del BIOS (0040h:006Ch) o esperando el retrazo vertical (puerto 3DAh, bit 3).
RG-07	Paleta personalizada.	Al menos 16 entradas de la paleta se redefinen con OUT a los puertos 3C8h y 3C9h. Se debe entregar la tabla de colores usada.
RG-08	Dibujo sin parpadeo.	Se redibuja solo lo que cambia, o se usa un buffer intermedio de 64 000 bytes copiado a A000h con REP MOVSW. El tablero no debe parpadear al moverse la selección.

Formato de Sprite

Un sprite es simplemente una tabla de bytes en el segmento de datos: un byte por píxel, con el índice de color de la paleta, recorrido de izquierda a derecha y de arriba hacia abajo. Un sprite de 16 × 16 ocupa 256 bytes. El valor 0 significa «transparente».
Se recomienda dibujar cada sprite en papel cuadriculado o en un editor de píxeles y generar la tabla con un pequeño script auxiliar (Python, JavaScript, lo que prefieran). El script auxiliar es una herramienta de diseño, no forma parte del programa entregado, pero debe adjuntarse en la carpeta /herramientas.

Personalización temática

La temática de referencia de este enunciado es «Toros y Vacas»: las fichas sus cabezas de distintas razas, la retroalimentación se muestra con un icono de toro y uno de vaca, y la mascota de la pantalla de presentación es un toro que resopla. Ustedes deben proponer otra, siempre que se mantenga la equivalencia conceptual, cada uno de los proyectos debe tener una temática diferente.

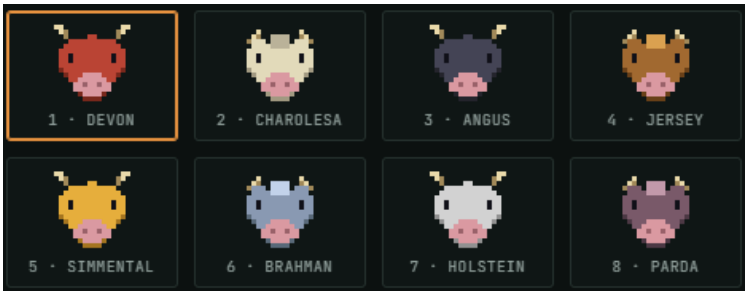


Figura 3: Sprites

Concepto del juego	Tema «Toros y Vacas»	Ejemplo de tema alternativo
Ficha / color	Cabeza de res de una raza	Especie de pez, planeta, fruta, nave
Acierto exacto (toro)	Icono de toro	Pez capturado, planeta colonizado
Acierto parcial (vaca)	Icono de vaca	Pez que escapa, planeta avistado
Mascota animada	Toro que resopla	Cualquier personaje propio

No se permite usar sprites tomados de videojuegos comerciales, personajes con derechos de autor ni imágenes descargadas de internet. Los sprites deben ser diseño original de la pareja; esto es parte de lo que se evalúa.

Requisitos técnicos de ensamblador

ID	Requisito	Criterio de aceptación
RT-01	Programa modular.	Mínimo 12 procedimientos con responsabilidad única. Ningún procedimiento supera las 200 líneas.
RT-02	Paso de parámetros documentado.	Cada PROC declara en su encabezado qué recibe, qué devuelve y qué registros destruye.
RT-03	Preservación de registros.	Los procedimientos preservan con PUSH/POP los registros que modifican y no requieren devolver. La pila queda balanceada.
RT-04	Uso de al menos una macro.	Definida con MACRO/ENDM y utilizada más de una vez.
RT-05	Datos en tablas, no en constantes dispersas.	El código secreto, el intento actual y el historial se almacenan en arreglos accedidos con direccionamiento indexado o base-indice.
RT-06	Comentarios significativos.	Se comenta la intención, no la instrucción. «mov ax,0A000h; ES apuntará a la VRAM» sí; «mov ax,0A000h; mueve a ax» no.
RT-07	Ensamblado limpio.	El proyecto ensambla y enlaza sin errores ni advertencias con el comando documentado en el README.
RT-08	Sin dependencias de alto nivel.	No se permite incrustar código C, ni usar librerías gráficas externas. Se permiten las macros y librerías del curso si el docente lo autoriza por escrito.

Restricciones

- Prohibido usar un lenguaje distinto al ensamblador para la lógica o el dibujo del juego.
- Prohibido entregar código descargado de repositorios de Mastermind en ensamblador. La revisión incluye búsqueda de coincidencias.
- Prohibido usar herramientas de IA para generar el código del proyecto sin declararlo; véase la sección 10.
- El programa debe ejecutarse en DOSBox con la configuración estándar del curso. «En mi máquina sí corre» no es un criterio de evaluación.

Maqueta de diseño

Esta sección define la disposición mínima de pantalla que debe respetar el proyecto. El estilo visual es libre; la organización de las zonas no lo es, porque asegura que todas las entregas sean comparables durante la revisión.



Figura 4: Juego



Figura 5: Fin del juego

Sistema de coordenadas

En modo 13h la pantalla es un arreglo lineal de 64 000 bytes que comienza en A000h:0000h. El píxel (x, y) está en el desplazamiento:

```
offset = y * 320 + x

; Ejemplo: píxel (30, 24) —- 24*320 + 30 = 7710
mov ax, 0A000h
mov es, ax ; ES —- memoria de video
mov ax, 320
mul coordY ; DX:AX = y * 320
add ax, coordX
mov di, ax ; ES:DI -¿píxel destino
mov BYTE PTR es:[di], indiceColor
```

Zonas de la pantalla

Zona	Origen (x, y)	Tamaño	Contenido
A. Encabezado	(0, 0)	320 × 20	Título del juego y nivel de dificultad activo.
B. Tablero	(4, 22)	196 × 168	Historial de intentos: 10 filas de 16 px de alto.
C. Panel lateral	(204, 22)	112 × 168	Paleta de fichas, intento en construcción, contador e imagen animada.
D. Barra de estado	(0, 190)	320 × 10	Ayuda de teclas y mensajes de validación.



Figura 6: Zonas de pantalla

Entregables

Nº	Entregable	Detalle
E1	Código fuente	Archivos .asm y .inc comentados. Se entrega en un repositorio o comprimido con la estructura: /src, /bin, /sprites, /herramientas, /doc.
E2	Ejecutable	Archivo .exe o .com ensamblado y probado en DOSBox, con el archivo .conf o el script de arranque usado.
E3	README	Instrucciones exactas de ensamblado, enlazado y ejecución. Debe funcionar copiando y pegando los comandos.
E4	Documento técnico	De 8 a 12 páginas: diseño de la solución, diagrama de módulos, explicación del algoritmo de toros y vacas, mapa de memoria de video utilizado, tabla de la paleta, hoja de sprites, dificultades encontradas y cómo se resolvieron.
E5	Hoja de sprites	Imagen con todos los sprites y sus cuadros de animación, más las tablas de bytes correspondientes.
E6	Bitácora de trabajo	Una página por semana indicando qué hizo cada integrante. Se usa para diferenciar notas individuales si es necesario.
E7	Video demostrativo	De 3 a 5 minutos mostrando una partida completa ganada y una perdida. Opcional pero recomendado.

Evaluación

Rubro	Peso	Forma de evaluación
A. Lógica y funcionamiento	50 %	Ejecución guiada frente al docente.
B. Gráficos, sprites y personalización	20 %	Inspección visual y revisión de código.
C. Defensa escrita	25 %	Prueba escrita individual el día de la defensa.
D. Documentación y proceso	5 %	Revisión de entregables E3 a E6.
Puntos extra	+5 %	Funcionalidad opcional de la sección 3.1.

Rubro A

Criterio	Puntos
Cálculo de toros y vacas correcto en los cuatro casos de la Tabla 2, incluyendo colores repetidos.	15
Generación del código secreto pseudoaleatoria y conforme al nivel.	5
Ciclo de juego completo: turnos, validación, historial, victoria, derrota, reinicio.	12
Manejo de teclado robusto: teclas extendidas, teclas inválidas ignoradas, sin cuelgues.	6
Menú y navegación entre pantallas funcionando en cualquier orden.	5
Calidad estructural del código: modularidad, pila balanceada, macros, tablas de datos.	5
Salida ordenada con restauración del modo de video y sin dejar la consola inutilizable.	2
Total	**50**

Rubro B

Criterio	Puntos
Rutina genérica de sprites con color transparente, usada por todo el juego.	5
Sprites propios, originales y distinguibles para todas las fichas y los iconos.	5
Animación efectiva en al menos tres elementos, con velocidad independiente de la CPU.	4
Paleta personalizada programada por puertos y documentada.	3
Ausencia de parpadeo y coherencia visual del conjunto (identidad temática lograda).	3
Total	**20**

Rubro D

Criterio	Puntos
Documento técnico completo y con contenido propio.	2
README reproducible: los comandos funcionan tal como están escritos.	1
Hoja de sprites y tabla de paleta entregadas.	1
Bitácora con reparto de trabajo verificable.	1
Total	**5**

Penalizaciones

Situación	Efecto
Entrega tardía	-10 % por día natural, hasta 3 días. Después no se recibe.
El proyecto no ensambla con los comandos del README	Se evalúa únicamente lo que el docente logre ejecutar; el rubro A se califica sobre lo demostrado.
Ausencia injustificada a la defensa	Nota 0 en el rubro C para el estudiante ausente.
Código no explicable por sus autores	Se anula el rubro correspondiente y se abre proceso por integridad académica.
Copia entre parejas o código de terceros sin declarar	Nota 0 en el proyecto para todas las partes involucradas.

Defensa

La defensa tiene dos partes que ocurren el mismo día: una demostración conjunta de la pareja y una prueba escrita individual. La prueba escrita existe porque el objetivo del curso no es que el juego funcione, sino que el estudiante entienda por qué funciona.

Parte práctica (no calificada aparte)

La pareja ejecuta el juego frente al docente durante 10 minutos y responde preguntas puntuales sobre el código en pantalla. Esta parte alimenta la calificación de los rubros A y B; el docente puede pedir modificaciones en vivo, por ejemplo: cambiar el color transparente, mover una zona de la pantalla o alterar la velocidad de la animación.

Parte escrita individual (25 %)

Aspecto	Detalle
Modalidad	Individual, sin computadora, sin apuntes. Se permite una hoja de referencia de una cara escrita a mano por el estudiante.
Duración	50 minutos.
Estructura	Se le informara al estudiante con anticipación. Al igual que las secciones evaluables, inicialmente es una sección con 10 preguntas de selección y una de desarrollo.
Regla clave	Las preguntas se formulan sobre el código que ustedes entregaron. Un integrante que no pueda explicar los módulos que dice haber escrito recibe una nota individual distinta a la de su pareja.

Integridad académica y uso de herramientas de IA

Está permitido usar asistentes de IA para entender conceptos, depurar errores y revisar código, con dos condiciones:

1. Declararlo en el documento técnico: qué herramienta se usó, para qué y en qué partes del proyecto.
2. Ser capaz de explicar en la defensa cualquier línea del proyecto como si la hubieran escrito ustedes, porque para efectos de la nota la escribieron ustedes.

No está permitido entregar código generado que sus autores no comprenden. La prueba escrita individual y las preguntas están diseñadas precisamente para detectar esa situación, y el resultado no es una nota baja: es un proceso por integridad académica